Memoria compartida distribuida
Mini-simulador a nivel de página
Dos procesos, páginas que migran y escrituras exclusivas

Armando Antonio Aguilar Rosales
Josue Daniel Rivera Alvarenga
Guillermo Alejandro Hernandez Dubon
Heber Josue Canales Henriquez
Mauricio Alejandro Contreras Montoya

Ingeniería Informática - Sistemas Operativos
Universidad Centroamericana José Simeón Cañas
Antiguo Cuscatlán, El Salvador
Catedrático: David Garcia Cerna

17 de octubre de 2026

El problema que resolvemos

- Cada proceso tiene su propia memoria: no ve directamente el arreglo del otro.
- Queremos consultar y modificar páginas como parte de un espacio lógico común.
- Los datos viajan por sockets; las reglas deciden quién puede usarlos.

Objetivo de la guía

Demostrar migración para lectura e invalidación antes de escribir.

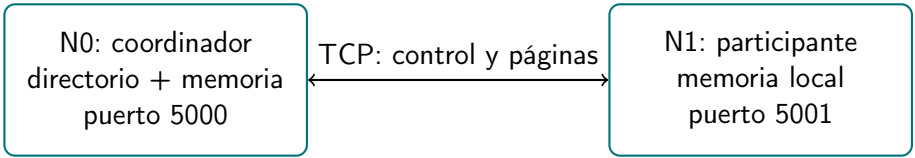
Qué construimos

- C11 en Pop!_OS / Linux.
- Dos procesos independientes.
- TCP IPv4: 127.0.0.1.
- Un hilo por proceso; poll.
- 1 MiB local por nodo.
- 256 páginas de 4096 bytes.
- Inicio: N0 dueño, N1 inválido.
- Fallo simulado al revisar etiquetas.

Alcance

No se interceptan punteros C ni fallos reales del kernel.

Una conexión y un coordinador



- N0 conecta; N1 acepta. Ambos pueden iniciar primero.
- El coordinador permanece en N0; la propiedad de datos puede migrar.

Tres etiquetas por página

INVALID	Puede haber bytes viejos, pero no se permite leerlos.
READ_ONLY	Copia válida para lectura; no autoriza modificar.
OWNER	Copia autoritativa; necesita exclusividad antes de escribir.

Dos lecturas distintas

`read`: migra y deja un dueño único.
`read-copy`: conserva una réplica para mostrar invalidación de lectores.

Demo 1: la página migra para leer

1. N0: `write 3 hola desde N0`.
2. N1: `read 3`; detecta INVALID y solicita la página.
3. Se preservan datos; N0 invalida su copia.
4. Viajan 4096 bytes; N1 instala, lee y confirma.

Comprobar en ambas terminales

`status 3`: N0 = INVALID; N1 = OWNER; versión = 1.

Demo 2: invalidar antes de escribir

1. N0: `read-copy 3`; queda READ_ONLY y N1 conserva OWNER.
2. N1: `write 3 actualizado por N1`; envía INVALIDATE.
3. N0 aplica INVALIDATE_APPLIED y concede exclusividad.
4. Solo entonces N1 escribe; versión = 2.
5. N0: `read 3`; recupera el texto nuevo y la propiedad.

Otra dirección

Si N0 escribe con lector N1, envía INVALIDATE y espera ACK remoto antes de escribir.

Por qué no se usan datos viejos

- Toda operación pasa por N0, incluso un acierto local.
- Se admite una transacción global: obtener datos, invalidar, conceder, confirmar.
- La copia INVALID nunca sirve una lectura.
- La escritura requiere confirmaciones; cada escritura aumenta la versión.
- Otra solicitud recibe BUSY y se reintenta manualmente.

Distinción

Coherencia trata las copias; consistencia trata el orden observable de accesos.

Transporte y errores

- TCP entrega bytes, no mensajes completos.
- Cabecera de 40 bytes y longitudes exactas; una página lleva 4096 bytes.
- Buffers conservan recepción y envío parciales; no bloquean la terminal.
- Se rechazan índices, tamaños, versiones o ACK fuera de fase.
- Plazos: saludo/config 10 s; operación/trama/cola 5 s.

Ante caída

Se aborta y se reinician ambos nodos. No se promete recuperación de propiedad.

Pruebas y resultados reales

- Compilación limpia sin warnings; tres programas de pruebas C.
- 22 pruebas de integración aprobadas con procesos TCP reales.
- Nueve escenarios DSM con ASan/UBSan, sin errores detectados.
- ACK retrasado: se comprueba que la escritura espere y que otra solicitud reciba BUSY.
- 48 operaciones con semilla fija contrastadas con un modelo de valores.

Demo de cinco operaciones

3 transferencias de página = 12288 bytes de payload; 3 fallos simulados; 3 invalidaciones aplicadas; versión final 2; propietario final N0.

Reproducir desde la terminal

```
make
# Terminal N0
./build/dsm-node --id 0 --port 5000 --peer-port 5001
# Terminal N1
./build/dsm-node --id 1 --port 5001 --peer-port 5000
```

Esperar DSM_READY. Introducir una orden y esperar su resultado.

```
make test      # pruebas
make demo      # demo y trazas verificadas
make package   # fuentes + articulo y presentacion PDF
```

Límites, aprendizaje y fuentes

- Dos nodos, memoria volátil, loopback y una operación a la vez.
- Sin acceso transparente, persistencia, reconexión ni tolerancia a fallos.
- Las pruebas dan evidencia; no son una demostración formal exhaustiva.
- Aprendizaje: relacionar páginas, propiedad, mensajes y orden de acceso.

Referencias: Li y Hudak, ACM TOCS 7(4), 1989; Lamport, IEEE TC C-28(9), 1979; Linux man-pages: tcp(7), recv(2), send(2), poll(2). Enlaces en el artículo y FUENTES.md.